

Java Arithmetic Operators & Integer Division

Learning Objectives

After this topic, you will be able to:

- Use Java arithmetic operators.
 - Understand how arithmetic expressions work.
 - Understand integer division.
 - Avoid common arithmetic mistakes.
 - Solve interview and output-based questions.
-

1. Arithmetic Operators

Java provides the following arithmetic operators:

Operator	Meaning	Example
+	Addition	5 + 3
-	Subtraction	5 - 3
*	Multiplication	5 * 3
/	Division	10 / 2
%	Modulus (Remainder)	10 % 3

2. Basic Examples

```
int a = 10;
int b = 3;

System.out.println(a + b); // 13
System.out.println(a - b); // 7
```

```
System.out.println(a * b); // 30
System.out.println(a / b); // 3
System.out.println(a % b); // 1
```

3. Integer Division

When **both operands are integers**, Java performs **integer division**.

The decimal part is **discarded**, not rounded.

Example:

```
System.out.println(10 / 3);
```

Output

```
3
```

Because:

```
10 ÷ 3 = 3.333...
```

```
Integer Division → 3
```

4. Getting Decimal Results

If **at least one operand is a floating-point type**, Java performs floating-point division.

```
System.out.println(10.0 / 3);
```

Output

```
3.3333333333333335
```

Or:

```
int a = 10;  
int b = 3;  
  
System.out.println((double) a / b);
```

Output

```
3.3333333333333335
```

5. Modulus (%) Operator

The % operator returns the **remainder** after division.

```
System.out.println(10 % 3);
```

Output

```
1
```

Common uses:

- Check even/odd numbers
- Cyclic operations
- Loop patterns

Example:

```
if (num % 2 == 0) {  
    System.out.println("Even");  
}
```

```
}
```

6. Operator Precedence

Java follows **BODMAS/PEMDAS** rules.

1. ()
2. *, /, %
3. +, -

Example:

```
System.out.println(2 + 3 * 4);
```

Output

```
14
```

Because multiplication happens first.

7. Common Mistakes

✗ Expecting decimal output from integer division.

```
System.out.println(5 / 2);
```

Output

```
2
```

Not 2.5.

✓ Correct:

```
System.out.println(5.0 / 2);
```

Output

```
2.5
```

✗ Dividing by zero.

```
int x = 10 / 0;
```

Result

```
ArithmeticException: / by zero
```

8. Best Practices

- Use double when decimal precision is required.
- Remember: int / int always gives an int.
- Use % to check divisibility and even/odd conditions.
- Use parentheses to make complex expressions clear.

9. Tricky Interview Questions

Q1. What is integer division?

Answer: Division where both operands are integers. The fractional part is discarded.

Q2. What is the output?

```
System.out.println(7 / 2);
```

Answer:

```
3
```

Q3. How do you get 3.5 instead of 3?

Answer:

```
System.out.println(7.0 / 2);  
// or  
System.out.println((double) 7 / 2);
```

Q4. What does % return?

Answer: The remainder after division.

Q5. What happens when dividing an integer by zero?

Answer: An ArithmeticException is thrown at runtime.

10. Output-Based Questions

Question 1

```
System.out.println(10 / 3);
```

Output

```
3
```

Question 2

```
System.out.println(10 % 3);
```

Output

```
1
```

Question 3

```
System.out.println(10.0 / 3);
```

Output

```
3.3333333333333335
```

Question 4

```
System.out.println(2 + 3 * 4);
```

Output

```
14
```

Question 5

```
System.out.println((2 + 3) * 4);
```

Output

20

11. Cheat Sheet

Expression	Output
10 + 5	15
10 - 5	5
10 * 5	50
10 / 5	2
10 / 3	3
10.0 / 3	3.3333333333333335
10 % 3	1
5 % 2	1
6 % 2	0

Key Points to Remember

- +, -, *, /, and % are Java's arithmetic operators.
- **int / int always results in integer division.**
- The decimal part is **truncated**, not rounded.
- Use double or cast one operand to double for decimal results.
- % returns the remainder and is commonly used to check divisibility and even/odd numbers.
- Dividing an integer by zero throws an ArithmeticException.